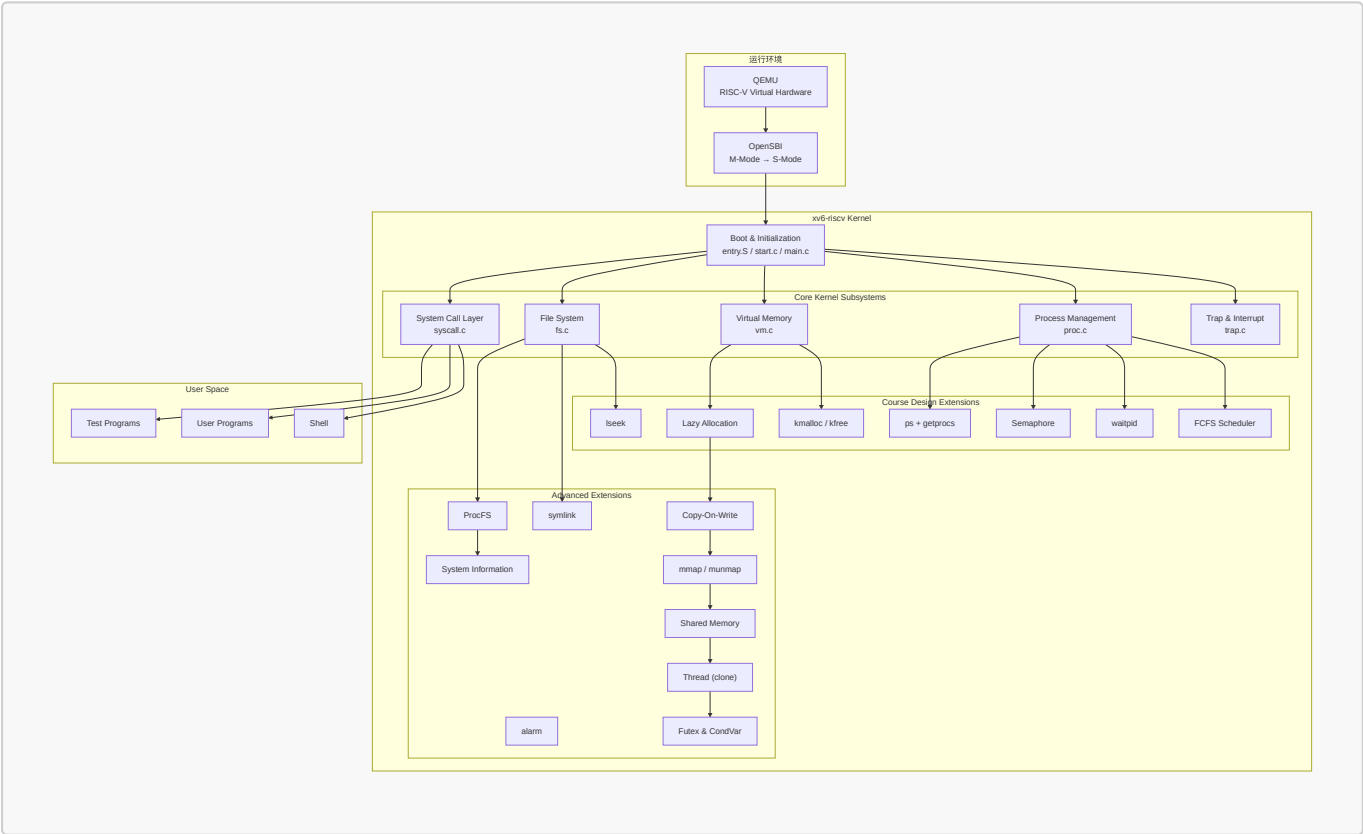


# 进度汇报

- 姓名：袁善
- 学号：20231072030
- 邮箱：shanyuan.dlut@gmail.com
- 日期：2026年6月6日

## 一、项目摘要

- 课设选题：方案 A：OS 内核实现
- 基准：基于 MIT `xv6-riscv`(<https://github.com/mit-pdos/xv6-riscv>)，代码基线回退至 2023 年 1 月前的稳定状态。
- 目标：在完成课程要求功能的前提下，引入部分现代 Unix/Linux 内核设计思想，提高系统的完整性与可扩展性。
- 开发模式：采用 Windows (VSCode) + 远程连接 (SSH) + 虚拟机 (Ubuntu 22.04) + 模拟器 (QEMU) 开发架构。
- 仓库地址：<https://github.com/yuuichi33/OS> 目前是 private 状态，验收前会改成 public 状态。
- 交付物：可运行源代码 + 技术文档 + 演示文稿 + 视频（对照课程评分标准）



## 二、技术方案

### 2.1 系统设计目标

xv6 是一个面向教学的 Unix 风格操作系统，其代码结构清晰、模块划分合理，完整实现了进程管理、虚拟内存管理、文件系统、系统调用和异常处理等核心机制。

本项目拟在保持 xv6 原有体系结构稳定性的前提下，逐步扩展其功能，实现课程设计要求的操作系统关键机制，并在此基础上引入部分现代 Unix/Linux 内核设计思想，提高系统的完整性与可扩展性。

重点参考 Linux 和开源项目 Re-XVapor (<https://github.com/sandyyyzy/Re-XVapor>) 以及 MIT 6.S081。

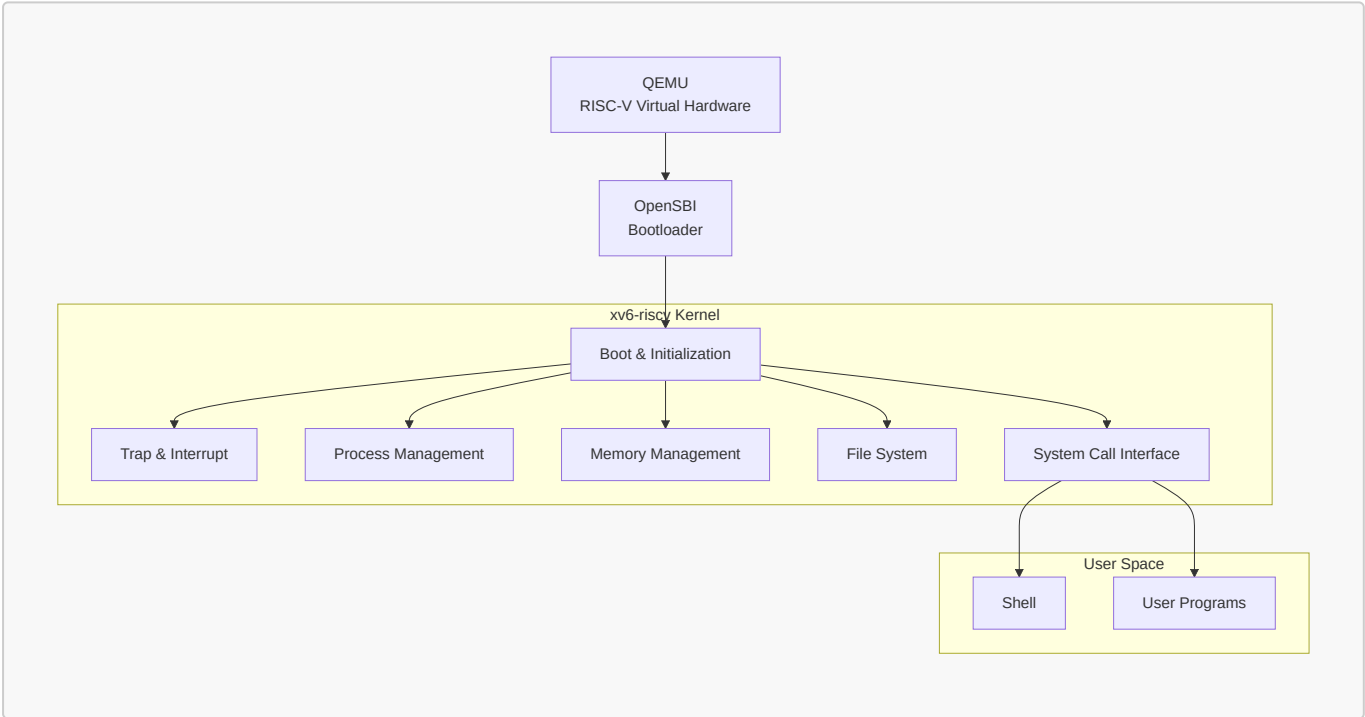
系统设计目标如下：

- 熟悉 xv6-riscv 内核整体架构与启动流程；
- 掌握操作系统核心子系统的实现原理；
- 完成课程要求的内存管理、进程管理、文件管理等功能扩展；
- 完成额外增加的扩展功能；
- 建立统一的系统调用与资源管理框架；
- 构建可持续扩展的实验操作系统平台；
- 构建完整的测试与验证体系。

最终形成一个具备较完整内核功能的增强型 xv6 操作系统。

2.2 系统总体架构

系统采用经典的分层式设计。整体运行环境由 QEMU、OpenSBI 和 xv6 内核组成。系统总体架构如图所示。



- QEMU 提供 RISC-V 虚拟硬件环境；
- OpenSBI 完成机器态初始化和特权级切换；
- xv6 内核负责资源管理和系统服务；
- 系统调用层提供用户态与内核态交互接口；
- 用户空间运行 Shell 和各类应用程序。

2.3 基于课程大纲子系统设计

1. 系统启动（Bootloader）

- 课程大纲要求

- **功能示例**：实现从实模式到保护模式（x86）或从 M 态到 S 态（RISC-V）的切换；加载内核镜像到内存正确位置并跳转执行；初始化段描述符表（GDT/IDT）或中断向量表；设置栈指针，为 C 语言运行环境做准备；输出启动日志信息，证明启动成功。
- **技术指标**：能在模拟器中成功启动并进入内核主函数；启动过程稳定可复现，无异常重启或死机。

- xv6 现状与开发规划

- **现状**：xv6 底层配合 OpenSBI 已完整实现从 M 态到 S 态的平滑切换。在 `entry.S` 中安全初始化了各个 CPU 核的内核栈指针，并在 `start.c` 中配置了必要的 CSR 寄存器，稳定调入 `main.c` 主函数。最终进入 Shell，系统开始正常运行。
- **规划**：本项目将完全复用该底层机制。后续开发过程中将在启动阶段增加调试日志输出，用于展示系统初始化过程和资源状态。

## 2. 中断与异常处理

- 课程大纲要求

- **功能示例**：实现中断描述符表（IDT）的初始化与注册；实现时钟中断处理程序，支持定时功能；实现键盘中断处理，支持基本输入；实现系统调用接口（`int 0x80` 或 `ecall`），支持用户态到内核态切换；实现缺页异常处理。
- **技术指标**：时钟中断频率稳定（如 100Hz）；键盘能正确读取扫描码并转换为 ASCII 字符；系统调用支持打印、进程创建、进程退出、文件读写。

- xv6 现状与开发规划

- **现状**：xv6 采用基于 Trap 的统一异常处理框架。所有系统调用、异常和中断最终均通过 Trap 机制进入内核。xv6 已通过 `stvec` 寄存器及 `trampoline.S/trap.c` 构建了完整的中断向量表与异常分发路由；时钟中断稳定运行；串口驱动支持键盘扫描码转换 ASCII 码输入；`ecall` 原生支持 `write`、`fork`、`exit`、`read` 等核心系统调用。
- **规划**：后续计划在现有框架基础上扩展：缺页异常处理；用户态非法访问检测；更完善的异常诊断信息输出。

## 3. 内存管理

- 课程大纲要求

- **功能示例**：实现物理内存的探测与管理（可用内存区域识别）；实现物理页框分配器（基于位图或链表的伙伴算法/简单分配器）；实现虚拟内存管理：页表创建、映射、解除映射；实现内核堆内存分配（`kmalloc/kfree`）；实现用户态内存空间布局（代码段、数据段、堆、栈）。
- **技术指标**：支持至少 4MB 物理内存管理；页大小为 4KB；支持按需分页（Demand Paging）；内存分配无泄漏，支持释放后重用。

- xv6 现状与开发规划

- **现状**：xv6 当前采用页式内存管理机制。底层物理页分配器由 `kalloc/kfree` 实现。xv6 已实现物理内存映射（默认 128MB）；页框分配器采用简单的 4KB 空闲链表（`kalloc.c`）；虚拟内存使用 Sv39 标准三级页表实现映射与解映射；用户态空间布局原生支持良好。
- **规划**：
  1. **内核堆内存管理**：在页级分配器基础上设计字节级动态分配器，实现 `kmalloc/kfree` 用于支持内核对象动态申请与释放。
  2. **实现按需分页（Demand Paging）**：改造用户堆空间增长方式。用户申请内存时仅扩展虚拟地址空间，在实际访问时通过缺页异常完成物理页分配，从而减少无效内存占用，达到 Lazy Allocation 的技术指标。

## 4. 进程管理

- 课程大纲要求
  - 功能示例：实现进程控制块（PCB）数据结构；实现进程创建（fork/exec）；实现进程调度器（支持至少两种算法：FCFS 和 RR）；实现进程状态转换：就绪、运行、阻塞、僵尸；实现进程间同步机制：信号量、互斥锁；实现进程退出与等待（wait/waitpid）。
  - 技术指标：支持多个并发进程；时间片轮转调度时间片可配置（默认 10ms）；进程切换开销小于 1ms（在 QEMU 中）；支持父子进程关系树。
- xv6 现状与开发规划
  - 现状：xv6 拥有完整的 PCB 结构 `struct proc`、并发进程支持、多核状态机转换以及 `wait` 机制，默认调度为 RR（10ms 时间片）
  - 规划：
    1. 实现 **FCFS**：增加 FCFS 调度模式。支持 RR 与 FCFS 两种调度算法切换。
    2. 进程同步机制：基于自旋锁 `spinlock` 和 `sleep/wakeup` 原语，封装标准的信号量控制结构，实现原生 P/V 操作与互斥锁功能。
    3. 实现 **waitpid** 机制：支持父进程等待指定子进程退出。

## 5. 文件系统

- 课程大纲要求
  - 功能示例：实现简化版文件系统（如基于内存的 RAMFS 或简化 EXT2）；支持文件/目录的创建、删除、打开、关闭、读写；实现文件描述符表管理；路径解析（绝对路径与相对路径）；支持标准输入输出重定向。
  - 技术指标：支持至少 128 个文件，单文件最大 64KB；目录层级至少支持 3 层嵌套；文件读写支持 seek 操作；提供 `mkfs` 工具初始化文件系统镜像。
- xv6 现状与开发规划
  - 现状：xv6 使用日志型文件系统。主要由 Buffer Cache、Logging Layer、Inode Layer、Directory Layer 组成。原生的 Inode 日志文件系统已具备缓存层、Logging 日志层、完备的文件描述符表及路径解析（`namei`），宿主机 `mkfs` 工具完备。
  - 规划：重点实现 `lseek` 操作，完成对文件定位功能的支持。

## 6. 用户程序加载与执行

- 课程大纲要求
  - 功能示例：实现 ELF 格式可执行文件的解析与加载；实现用户态栈初始化（参数传递、环境变量）；实现系统调用封装库（libc 简化版）；实现 shell 命令行解释器，支持基本命令（ls, cat, echo, ps, kill, exec）。
  - 技术指标：至少能加载运行 5 个不同的用户程序；shell 支持命令解析、参数传递、管道；用户程序崩溃不影响内核稳定性。
- xv6 现状与开发规划
  - 现状：`exec.c` 支持 ELF64 加载与用户栈传参配置；内置的 Shell 支持管道、后台运行（`&`）等。
  - 规划：
    1. 编写系统状态程序 `ps.c`：在内核中增加 `sys_getprocs` 系统调用，用于将进程表中活跃进程的状态、名称、PID 和父进程信息安全拷贝至用户态，再在用户态 `ps.c` 中进行格式化输出。
    2. 确保在 `usertrap` 中正确拦截由于用户态程序引发的非法内存访问和非法指令执行等异常事件。在异常发生时，由内核强制终止（Kill）该出错的用户进程并调用 `exit` 回收其所有

资源，确保用户程序崩溃时，内核不会 Panic，整体系统持续稳定。

## 2.4 额外扩展功能设计

除课程设计要求外，计划进一步扩展以下高级功能（可选），逐步向现代 Unix/Linux 内核设计靠拢。

- alarm
- symlink
- Copy-On-Write Fork
- mmap/munmap
- Shared Memory
- 多线程机制
- Futex 和条件变量
- ProcFS
- system information

## 2.5 测试方案

- 单元测试：针对新增功能分别自行设计测试程序，精准验证单一模块功能的正确性。
- 异常测试：构造非法内存访问和异常系统调用场景，验证系统异常隔离能力和资源回收能力。
- 系统测试：运行 xv6 原生测试集 `usertests`，全面验证进程管理、内存管理、文件系统及系统调用等核心功能的正确性与兼容性。
- 集成测试：实现统一测试框架 `alltests.c`，对新增功能测试、异常测试以及 xv6 原生 `usertests` 进行统一调度，实现一键式自动化测试。通过集成运行验证各模块之间的兼容性与协同工作能力，并检查系统整体稳定性。

## 2.6 开发过程中想到的其他内容

- FCFS RR (SJF NP-FP)
- 增加图形化界面

# 三、任务清单及进度规划

- 阶段一：基础环境与系统分析（Unchanged）
  - ☒ 完成 QEMU、GCC 交叉工具链及 SSH 远程开发环境部署。
  - ☒ xv6 架构与启动流程分析。
  - ☒ 跑通 `usertests` 基准测试，建立 Baseline。
- 阶段二：系统调用、异常防护与动态内存（Foundation）
  - ☒ 异常防护机制：在 `usertrap` 中拦截非法地址/指令，确保用户态崩溃不影响内核。
  - ☒ `kmallocl/kmfree`：实现内核级字节动态分配器，为后续的信号量、定时器、线程等提供动态内存支持。
  - ☒ 实现 `getprocs` 系统调用与用户态 `ps` 程序（先使用系统原生的静态进程表）。
- 阶段三：核心进程管理、调度与同步（Core Process & Sync）
  - ☒ FCFS 调度器：引入创建时间戳，实现非抢占 FCFS 与 RR 的动态切换。
  - ☒ `waitpid`：扩展进程回收机制，支持回收指定子进程。

- ☒ Semaphore (信号量): 基于自旋锁与 sleep/wakeup 实现, 由于有了 kmalloc, 此时可以优雅地实现 sem\_alloc/sem\_free。
- ☐ Alarm 异步事件通知: 基于时钟中断、Trapframe 现场保存与恢复实现定时通知。
- 阶段四: 文件系统增强 (File System)
  - ☐ lseek: 实现文件指针定位, 支持 SEEK\_SET/CUR/END。
  - ☐ Symlink (软链接): 实现符号链接节点, 并在 namei 路径解析中引入递归解析与死循环防御。
- 阶段五: 进阶虚拟内存管理 (Advanced VM)
  - ☐ Lazy Allocation (按需分页): 重构 sbrk, 通过捕获 13/15 号缺页中断动态分配物理页。
  - ☐ Copy-On-Write Fork (写时复制): 在 kalloc 中引入物理页引用计数, 在 fork 时共享只读页表, 写操作时触发缺页拷贝。
  - ☐ mmap/munmap: 引入虚拟内存区域 (VMA) 管理, 实现文件与匿名的内存映射。
  - ☐ Shared Memory (共享内存): 基于 VMA 和引用计数, 实现多进程共享物理页。
- 阶段六: 多线程机制与用户态同步 (Threading & Futex)
  - ☐ clone: 利用阶段五建立起来的成熟页表管理机制, 通过 uvmshare 共享物理地址空间, 为轻量级线程创建独立的页表、Trapframe 和用户栈。
  - ☐ Futex 和条件变量
- 阶段七: 系统信息与虚拟文件系统 (Virtual FS)
  - ☐ ProcFS 虚拟文件系统: 实现动态虚拟 Inode 映射机制。
  - ☐ System Information: 实现 /proc/meminfo 和 /proc/[pid]/status, 将阶段二的 getprocs 和阶段五的 VMA 状态以虚拟文件形式直观暴露。
- 阶段八: 测试与全量验证 (Testing)
  - ☐ 模块单元测试
  - ☐ 集成测试与原生 usertests 压力测试
  - ☐ 一键自动化测试框架 alltests 跑通

## 四、具体完成工作

### 4.1 基础环境与系统分析

- 构建 Windows 11 (VSCode) + SSH 远程连接 + Ubuntu 22.04 虚拟机的开发架构。
- 完成 RISC-V 交叉编译器 (gcc-riscv64)、调试器与 QEMU 模拟器的配置。
- 初始化 GitHub 仓库 <https://github.com/yuuichi33/OS> 并完成首次推送。
- 新建并切换至独立开发分支 dev, 强制回滚至 2022 年底稳定提交 74c1eba, 避开后续版本对 QEMU >= 7.2 的编译限制, 确保兼容性。
- 通过 make qemu 成功编译并启动系统。

- 在模拟器中全量跑通内核测试集 `usertests`，测试结果为 `ALL TESTS PASSED`。

```
yuans@yuans-virtual-machine:~/os/xv6-riscv$ make qemu
test badarg: OK
usertests slow tests starting
test bigdir: OK
test manywrites: OK
test badwrite: OK
test execout: OK
test diskfull: balloc: out of blocks
balloc: out of blocks
OK
test outofinodes: ialloc: no inodes
OK
ALL TESTS PASSED
$
```

## 4.2 系统调用、异常防护与动态内存

- 分析 (`devlog/phase2.md`)
- 用户态异常捕获
  - 修改 `trap.c` 中的 `usertrap()`，实现对非法指令 (`scause 2`) 与内存越界读写 (`scause 13/15`) 的分类识别。
  - 发生异常时，内核打印错误地址与指令并强制结束该进程 (`exit(-1)`)，保证内核和其他进程正常运行不崩溃。
- 集成测试框架 (`alltests`)
  - 编写 `alltests.c` 自动测试程序，通过 `fork` 一键运行所有测试并比对退出状态码。
  - 一键跑通了包含正常调用、非法指令、非法读写在内的全部 4 个测试用例 (`PASS: 4/4`)。
- `getprocs` 系统调用
  - 结构体 `struct uproc` 用于在内核与用户态间传递 `PID`、状态、内存大小及进程名。
  - 实现 `sys_getprocs`，遍历全局进程表，通过 `copyout` 安全地将打包数据拷贝至用户空间。
- 用户态 `ps` 命令
  - 编写 `ps.c` 工具，打印进程状态。
  - 将 `ps` 接入测试框架 `alltests`，测试结果全量通过 (`PASS: 5/5`)。
- 内核动态内存分配器 (`kmalloc/kmfree`)
  - 基于首部链表 (`First-Fit Header`) 实现。申请时自动进行 8 字节对齐并按需拆分空闲块；无可用块时，向底层页分配器索要全新物理页。
  - 在 `kmfree` 中实现了物理连续空闲块的自动检测与合并，有效防止了内存碎片的产生。



- 使用独立的自旋锁保护分配链表，保证了多 CPU 并发分配下的数据安全。
- 编写 `kmalloctest.c` 单元测试，并将其接入 `alltests` 集成测试框架，测试顺利通过 (PASS: 6/6)。

```
yuans@yuans-virtual-machine:~/os/xv6-riscv$ make qemu

[Kernel Exception] Process 4 (crash_test) killed due to: Illegal Instruction
      at PC: 0x0000000000000018, Instruction: 0x0000000000000000
[PASS] Exception: Illegal Instruction (Expected crash, exit status: -1)

[RUN] Exception: Invalid Read...
Triggering: Invalid Read (Segmentation Fault)...

[Kernel Exception] Process 5 (crash_test) killed due to: Segmentation Fault (Invalid Read)
      at PC: 0x0000000000000040, Accessing Address: 0x0000007fffffffffff
[PASS] Exception: Invalid Read (Expected crash, exit status: -1)

[RUN] Exception: Invalid Write...
Triggering: Invalid Write (Segmentation Fault)...

[Kernel Exception] Process 6 (crash_test) killed due to: Segmentation Fault (Invalid Read)
      at PC: 0x00000000000000a2, Accessing Address: 0x0000007fffffffffff
[PASS] Exception: Invalid Write (Expected crash, exit status: -1)

[RUN] Exception: Write to Code...
Triggering: Write to Read-Only Code Segment...

[Kernel Exception] Process 7 (crash_test) killed due to: Segmentation Fault (Invalid Write)
      at PC: 0x00000000000000ea, Accessing Address: 0x0000000000000020
[PASS] Exception: Write to Code (Expected crash, exit status: -1)

[RUN] System call: ps...
PID   STATE   SIZE    NAME
1     SLEEPING 16384    init
2     SLEEPING 20480    sh
3     SLEEPING 16384    alltests
8     RUNNING  16384    ps
[PASS] System call: ps (Success, exit status: 0)

[RUN] Memory: kmalloc/kmfree...
kmalloc test: starting...
kmalloc test: all branches passed!
kmalloctest: OK
[PASS] Memory: kmalloc/kmfree (Success, exit status: 0)

=====
  ALL TESTS PASSED SUCCESSFULLY! (PASS: 6/6)
=====
$ █
```

### 4.3 核心进程管理、调度与同步

- [分析](#) (`devlog/phase3.md`)
- FCFS 调度器
  - 创建 `ctime` 时间戳，在 `proc.c` 的 `scheduler()` 中实现 FCFS 策略，调度时选取 `ctime` 最小（最早创建）的就绪进程。
  - 修改 `trap.c` 实现非抢占的 FCFS。
  - 新增 `sched_switch` 系统调用和 `sche`` 命令，支持切换调度模式。
  - 编写 `schedtest` 并接入 `alltests`。FCFS 模式下子进程完全顺序执行；RR 模式下子进程交替并发（打印交错）。测试结果全量通过 (PASS: 10/10)。



```

yuans@yuans-virtual-machine:~/os/xv6-riscv$ make qemu
Process 2 compute progress: 100%
Process 2 finished.
Process 3 compute progress: 100%
Process 3 finished.
schedtest: finished.
[PASS] Sched: FCFS scheduling (Success, exit status: 0)

[RUN] Sched: Switch to RR...
sched: switched to RR mode
[PASS] Sched: Switch to RR (Success, exit status: 0)

[RUN] Sched: RR scheduling...
Starting schedtest (spawning 3 compute processes)...
Process 1 compute progress: 20%
Process 2 compute progress: 20%
Process 3 compute progress: 20%
Process 1 compute progress: 40%
Process 2 compute progress: 40%
Process 3 compute progress: 40%
Process 2 compute progress: 60%
Process 3 compute progress: 60%
Process 1 compute progress: 60%
Process 2 compute progress: 80%
Process 3 compute progress: 80%
Process 2 compute progress: 100%
Process 2 finished.
Process 1 compute progress: 80%
Process 3 compute progress: 100%
Process 3 finished.
Process 1 compute progress: 100%
Process 1 finished.
schedtest: finished.
[PASS] Sched: RR scheduling (Success, exit status: 0)

=====
  ALL TESTS PASSED SUCCESSFULLY! (PASS: 10/10)
=====

```

- waitpid 机制

## 参考资料

- <https://github.com/mit-pdos/xv6-riscv>
- <https://github.com/mit-pdos/xv6-riscv-book/>
- <https://github.com/qemu/qemu>
- <https://github.com/sandyyyz/Re-XVapor>
- <https://github.com/tianx666/xv6-book-riscv-rev1-Chinese>

- [https://blog.csdn.net/zzy980511/category\\_11740137.html](https://blog.csdn.net/zzy980511/category_11740137.html)
- <https://github.com/mit-pdos/xv6-riscv-fall19>
- <https://github.com/torvalds/linux>
- <https://pdos.csail.mit.edu/6.S081>